Abstract Zero-Sum Deletion Complexity and Support-One Normalization in a Pauli Model

Sze Chun Yiu
sze-chun.yiu@fysik.su.se

25 August 2026

Abstract

An abstract support budget can be exact for its deletion language and still differ from the intrinsic support of a separately defined compiler model. We formalize this separation with an alphabet-restricted zero-sum invariant. For a finite abelian signature group H and allowed alphabet $A \subseteq H$, let $\mathrm{zsf}(H; A)$ be the largest length of a zero-sum-free word over A. Assume that A contains a nonzero element. In the deletion language whose only shortening step removes a nonempty proper zero-sum subsequence from a nonzero-total word, the exact uniform terminal complexity is $\mathrm{zsf}(H; A)$. Every longer word is reducible, while a longest zero-sum-free word is a matching terminal witness. A production compiler inherits the matching lower bound only if that witness is realizable and no additional rule can reduce it.

The abstract standard-basis alphabet in $\mathbb{F}_2^5$ has exact terminal complexity five. A two-block dependent-triple compiler using the same syndrome coordinates admits a whole-system Tag relocation with intrinsic support one. We do not assert that its production states realize the five-letter abstract terminal word. For t components, the separately defined certificate and product-support budgets are $5t$ and t. A direct support enumerator has ratio $\Theta(n^{4t})$ for fixed t under the stated model; this is no unrestricted proof or algorithm lower bound. A support number belongs to a compiler only after an intrinsic lower witness; otherwise it belongs to a named normalization or certificate language.

Keywords: Pauli compiler models; certificate complexity; support normalization; zero-sum deletion; exact optimization

1. Introduction

Support bounds turn unbounded exact compiler searches into finite ones. Their numerical value can nevertheless answer three different questions:

1. What support does the compiler intrinsically require?
2. What support can a particular normalization attain?
3. What support can a restricted proof language certify?

These quantities coincide in some families and differ sharply in others. The alphabet-Davenport normal-form theorem supplies a general upper certificate. Here we identify the exact complexity of its deletion language and compare it with independently established compiler support.

The distinction is operational. If a search implementation enumerates all supports up to a certificate ceiling, a loose proof language can impose a large, avoidable search volume even when the compiler

has a much smaller normal form. Conversely, a small normal form is not intrinsic until a lower witness rules out smaller support.

Our contributions are:

1. an exact terminal-complexity theorem for zero-sum deletion;
2. a realization criterion separating abstract terminal words from production compiler states;
3. a strict comparison between an exact abstract five-bit deletion language and intrinsic dependent-triple compiler support one; and
4. a direct-sum/product comparison with numerical difference $4t$ and a precisely scoped consequence for enumerators that adopt the abstract certificate.

2. Three support quantities

Fix a compiler family $\mathcal{F}$ and objective C. For every configuration considered here, *support* means the maximum Pauli weight of an independent frame generator. Auxiliary Tag strings contribute to the objective but not to this support functional. Product support is defined separately in Section 5.

2.1 Intrinsic support

The intrinsic support $\kappa(\mathcal{F}, C)$ is the least k such that every instance has an exact optimum of support at most k, together with an independent witness showing that support $k-1$ cannot always suffice.

2.2 Normalization ceiling

A transformation N may show that every feasible configuration has a no-more-expensive representative of support at most B_N. Without a matching compiler lower witness, B_N belongs to the triple $(\mathcal{F}, C, N)$, not intrinsically to $(\mathcal{F}, C)$.

2.3 Certificate complexity

A proof language P restricts visible information and legal inference. Write $\beta_P(\mathcal{F}, C)$ for the least uniform support budget that P can certify on its production scope. Soundness gives

$$\kappa(\mathcal{F}, C) \leq \beta_P(\mathcal{F}, C),$$

but equality requires additional mathematics.

Quantity	Owned by	Lower-witness requirement
Intrinsic support κ	compiler family and objective	compiler obstruction
Normalization ceiling B_N	compiler, objective, and transformation	only for intrinsic interpretation
Certificate complexity β_P	compiler scope and proof language	terminal witness realizable in production

3. Exact zero-sum deletion complexity

Let H be a finite abelian group and let $A \subseteq H$ contain at least one nonzero element. A certificate word $v_1 \cdots v_w$ has nonzero total. A subsequence is selected by an arbitrary set of positions and need not be contiguous. The only legal shortening removes a nonempty proper subsequence whose sum is zero. Define

$$\mathrm{zsf}(H; A) = \max\{|W| : W \text{ is zero-sum-free over } A\}.$$

Theorem 1 (exact abstract certificate complexity). The maximum terminal length among all nonzero-total words over A is exactly $\mathrm{zsf}(H; A)$.

Proof. A word longer than $\mathrm{zsf}(H; A)$ contains a nonempty zero-sum subsequence. Its nonzero total prevents that subsequence from being the whole word, so a legal deletion exists. Conversely, a longest zero-sum-free word has nonzero total and admits no legal deletion. □

Corollary 2 (production realization). Suppose every nonzero-total production word lies over A, and every legal abstract zero-sum deletion is an admissible, sound production move. Then the production ceiling is at most $\mathrm{zsf}(H; A)$. The ceiling is exact only if production realizes a longest zero-sum-free word and no other production rule reduces that state.

For $H = \mathbb{F}_2^d$, $\mathrm{zsf}(H; A) \leq d$; an alphabet containing a basis has equality.

4. Abstract certificate versus dependent-triple compiler

Define the abstract change alphabet directly by $A_5 = \{e_1, \ldots, e_5\} \subset \mathbb{F}_2^5$, where the e_i are the standard basis vectors. The word $e_1 \cdots e_5$ is zero-sum-free, and the binary rank bound gives $\mathrm{zsf}(\mathbb{F}_2^5; A_5) = 5$. Theorem 1 therefore gives the exact terminal complexity of this explicitly defined abstract five-bit deletion language,

$$\beta_{\text{abstract del}} = 5.$$

The Pauli-grammar comparison below uses the same five syndrome coordinates, but no claim is made that a single feasible production configuration realizes the abstract word $e_1 \cdots e_5$. Accordingly, we do not identify $\beta_{\text{abstract del}}$ with the production certificate complexity of the compiler.

The compiler family $\mathcal{C}_{\text{dep}}$ has two blocks $j \in \{A, B\}$, each with an anticommuting pair R_{j0}, R_{j1} and $R_{j2} = R_{j0}R_{j1}$. Each block may permute its three targets, and the two blocks share Tag strings S_0, S_1. With Pauli weight w, restored targets $T_{jk} = P_{j,\pi_j(k)}R_{jk}$, and multipliers $m_{jk} = 4$ except for the declared central case where $m_{jk} = 2$, the structural objective is

$$C = \sum_{j \in \{A,B\}} \left(\sum_{k=0}^{2} m_{jk} w(R_{jk}) - 10 \right) + 2(w(S_0) + w(S_1)) + \sum_{j,k} w(T_{jk}).$$

The shared Tag must assign three nonzero, distinct labels and must do so coordinatewise in both blocks: $c_{Ak} = c_{Bk}$ for every $k \in \{0, 1, 2\}$, where $c_{jk} = 2\langle S_0, R_{jk}\rangle + \langle S_1, R_{jk}\rangle$. These equations, the Pauli commutation constraints, and target permutations are the complete feasibility grammar used here.

A stronger transformation goes beyond rank-only deletion. We state every local premise because the support-one theorem must not depend on a private "parent'' calculation. Choose in each block a *core* column on which the two independent frames anticommute; such a column exists because their global symplectic product is one. Delete every non-core frame letter, recompute the dependent third frame, and place the shared Tags at the selected cores with canonical labels $(c_0, c_1) = (1, 2)$.

Lemma 3 (local reconstruction bounds). The following bounds hold for every local Pauli choice in the declared grammar.

1. At an active non-core column, deleting both independent frame letters and recomputing the dependent third frame changes frame-plus-Restore cost by at most -4 if the two letters commute and at most -7 if they anticommute.
2. Replacing one ordered anticommuting core basis by another leaves its weighted frame contribution equal to 10 and increases Restore cost by at most three.
3. Every feasible original shared Tag costs at least four. A canonical Tag at a common core costs four; canonical Tags at distinct cores cost eight.
4. If both blocks use the same core and already have support one, equal nonzero distinct shared labels force their ordered core bases to agree.
5. At distinct cores the minimum shared-Tag cost is eight for every pair of ordered anticommuting bases.

Proof. All statements are identities in the four-letter local Pauli alphabet. For item 1, direct substitution shows that a commuting active pair refunds at least six frame units while at most two Restore letters worsen; an anticommuting pair refunds 10 while at most three Restore letters worsen. For item 2, each ordered anticommuting basis and its dependent product has three nonidentity letters, so the central/noncentral weights sum to 10; only the three Restore letters can change. For item 3, the two prescribed nonzero distinct labels force both global Tag strings to be nonidentity, giving the original floor four, while the local dual bases realize the stated costs. Items 4 and 5 follow by solving the two symplectic label equations at one and two cores. The ancillary verifier exhausts 2,880 deletion cases, 6,912 core-alignment cases, 576 same-core label cases, and 9,216 distinct-core Tag cases; these enumerations check the finite identities and do not replace the composition argument. □

Theorem 4 (support-one normalization). Every feasible configuration of $\mathcal{C}_{\mathrm{dep}}$ has a no-more-expensive representative in which each frame generator has support at most one.

Proof. Choose one anticommuting core per block and perform the reconstruction above. The displayed objective is a sum over columns apart from the shared Tag weights, so item 1 of Lemma 3 applies to every deleted non-core column and the credits add. If the cores are distinct and both blocks already have support one, the old Tag cost is at least eight, which pays the new Tag. Otherwise a deleted non-core column supplies at least four units in addition to the original Tag floor four, again paying cost eight. If the cores coincide and their bases agree, the new Tag cost four does not exceed the old floor. If the bases differ, item 4 of Lemma 3 implies that a non-core column is deleted; its credit four exceeds the alignment penalty of at most three. No target, permutation, or central choice changes. The construction therefore preserves feasibility and never increases cost. □

Because the transformation removes every non-core frame letter, the resulting whole-system Tag relocation leaves each frame generator at support at most one. Support zero is infeasible because an identity frame cannot anticommute with its partner. Hence

$$\kappa(\mathcal{C}_{\mathrm{dep}}) = 1.$$

There is no contradiction: a basis word blocks zero-XOR deletion while the successful proof changes auxiliary Tag structure that the rank-only language holds fixed.

5. Disjoint product amplification

Let $\mathcal{C}_{\mathrm{dep}}^{\times t}$ be the disjoint product of t components on disjoint coordinates, with no cross-component move. Define its product support as the sum, over components, of the maximum Pauli support used by any frame in that component.

Theorem 5. For every $t \geq 1$,

$$\beta^{\oplus t}_{\text{abstract del}} = 5t, \qquad \kappa(\mathcal{C}_{\mathrm{dep}}^{\times t}) = t.$$

Proof. Direct-sum abstract certificate complexities add, and a basis word in each five-bit component gives the lower bound of five per component. Support zero is infeasible in every component, and the support-one normalization acts independently. Thus both lower and upper intrinsic bounds equal one per component. □

The numerical difference between these separately defined budgets is $4t$. This construction amplifies one mechanism; it is not evidence for a second, independent source of separation.

5.1 Consequence for direct support enumeration

For fixed t, fixed local alphabet, and fixed budget B, an enumerator that explicitly visits all coordinate supports of size at most B has leading growth $\Theta(n^B)$. If such an implementation adopts the direct-sum abstract certificate budget rather than the intrinsic compiler budget, the declared model yields

$$\Theta(n^{5t}) \quad \text{versus} \quad \Theta(n^t),$$

and therefore ratio $\Theta(n^{4t})$. This conditional bookkeeping statement does not establish that the five-bit abstract budget is necessary on the production compiler. It is not a complexity-class lower bound and does not constrain algorithms that avoid direct support enumeration.

6. Relation to prior work

Sparse integer optimization already studies support bounds and lower constructions [1]. Classical and modern zero-sum theory owns Davenport constants, restricted alphabets, weighted variants, basis obstructions, and direct sums [2,3]. Proof-complexity and formal-methods research already distinguishes object difficulty from lower bounds internal to a proof language.

The residual contribution is a calibrated comparison in explicit Pauli compiler models. The abstract five-bit deletion language is five times larger than the dependent-triple compiler's intrinsic support. Proving the same factor for the production certificate remains open. The latter normalization also identifies the missing proof operation—whole-system Tag reconstruction—instead of merely

reporting a smaller number. Pauli-based block compilers such as TARE [5] and Paulihedral [6] motivate the model class, but the present theorem is not claimed for either production system.

The companion normal-form paper [4] proves a deletion-dominance theorem and an arbitrary-block sufficient cone for a different multi-Tag grammar. The present paper neither reproves that cone nor uses it: its central results are the exact terminal complexity of a named proof language and the direct support-one normalization of $\mathcal{C}_{\mathrm{dep}}$. Conversely, the companion paper does not claim the proof-language separation established here.

7. Reproducibility and limitations

Finite-group controls, binary basis obstructions, production alphabets, and product formulas have been checked with a separate deterministic verifier. Theorems 1, 4, and 5 supply all-size authority.

The exact abstract theorem concerns only the stated deletion language. A production lower bound requires realization of its terminal witness. We prove no lower bound for every local, syndrome-preserving, or unrestricted proof system. The disjoint product forbids cross-component transformations by definition. The enumeration ratio belongs only to the direct support model. Finally, structural support does not imply a hardware speedup or any physical quantum-resource advantage.

8. Conclusion

A certificate can be sound, useful, and internally exact while measuring the wrong layer for an intrinsic interpretation. The alphabet-restricted invariant identifies the expressive ceiling of zero-sum deletion. The dependent-triple compiler shows that a whole-system transformation can cross it sharply.

The reporting rule is therefore simple: call a support number intrinsic only after an independent compiler lower witness. Otherwise identify the normalization or proof language that owns it.

Tool-use disclosure

A generative language model assisted manuscript organization, language revision, and submission-package preparation. The listed author remains responsible for the mathematical statements, proofs, references, executable claims, and final text.

Data and code availability

The source package contains result records for the change alphabet and a standalone verifier for the dependent-triple local lemmas and composition. Theorems 1, 4, and 5 carry the abstract, support-one, and product claims.

References

1. I. Aliev, J. A. De Loera, F. Eisenbrand, T. Oertel, and R. Weismantel, "The Support of Integer Optimal Solutions," *SIAM Journal on Optimization* **28**, 2152-2157 (2018). DOI: 10.1137/17M1162792

2. G. Wang, "The universal zero-sum invariant and weighted zero-sum for infinite abelian groups," *Communications in Algebra* **53** (2025). DOI: 10.1080/00927872.2024.2418017
3. M. Freeze and W. A. Schmid, "Remarks on a generalization of the Davenport constant," *Discrete Mathematics* **310**, 3373-3389 (2010). DOI: 10.1016/j.disc.2010.07.028
4. S. C. Yiu, "Zero-Sum Deletion Normal Forms for a Multi-Tag Pauli Grammar," companion manuscript (2026).
5. N. Schillo, A. Sturm, and R. Quay, "TARE: Block Encoding Linear Combinations of Pauli Strings Without Ancilla State Preparation," arXiv:2601.05740v4 [quant-ph] (2026).
6. G. Li, A. Wu, Y. Shi, A. Javadi-Abhari, Y. Ding, and Y. Xie, "Paulihedral: A Generalized Block-Wise Compiler Optimization Framework for Quantum Simulation Kernels," in *Proceedings of ASPLOS 2022*, 554-569 (2022). DOI: 10.1145/3503222.3507715